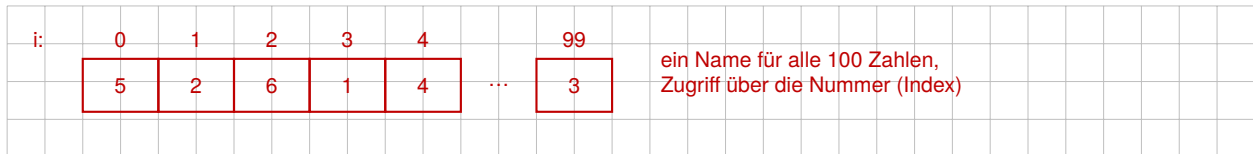


2.5 Verwaltung gleichartiger Objekte – Das Array / Feld

Aufgabe 1 Projekt Zufallszahlen

- a) Ergänze die Attribute zahl1 bis zahl3. b) Weise ihnen im Konstruktor Werte von 1 bis 6 zu.
c) Und bei 100 Zahlen? Skizziere, wie ein Speicher aussehen müsste, in dem du „die i-te Zahl“ ansprechen kannst:



Felder

Felder (engl. **Arrays**) dienen der Speicherung einer **festen Anzahl** von Werten **gleichen Datentyps**, die über einen Index (Positionsanzahl) adressiert werden können.

Dadurch kann z. B. mit einer Wiederholung auf alle Elemente zugegriffen werden, ohne jedes einzeln zu behandeln.

Index i	textFeld[i]
0	"Hallo "
1	"Welt, "
2	"ein "
3	"Satz."

Aufgabe 2

Übertrage den Code in das Projekt (TODO 2) und beschreibe die Funktion der Zeilen.

```
class Zufallszahlen {
    private int[] zahlenFeld;           // Deklaration: Feld für int-Werte

    Zufallszahlen() {
        zahlenFeld = new int[8];       // Erzeugung: 8 Zellen, alle mit 0 belegt
        zahlenFeld[0] = 5;             // erste Zelle (Index 0) bekommt 5
        zahlenFeld[7] = Random.randint(1, 6); // letzte Zelle bekommt Zufallszahl 1--6
    }
}
```

Merke

- Felder beginnen ihren Index immer bei **0**.
- Die Größe eines Feldes liefert `<Feldname>.length`. Hier also: **zahlenFeld.length**. Das Array ist **8** Zellen groß.
- Was passiert bei `zahlenFeld[8]`? **Laufzeitfehler (Index zu groß)**

Index i	zahlenFeld[i]
0	5
1	0
2	0
3	0
4	0
5	0
6	0
7	1 bis 6

Fülle die Tabelle auf Seite 1. Warum ist zahlenFeld[7] die letzte Zelle?

Der Index beginnt bei 0. Bei 8 Zellen laufen die Indizes von 0 bis 7, der letzte ist also $8 - 1 = 7$.

Letzte Zelle bei beliebiger Feldgröße: zahlenFeld[zahlenFeld.length - 1]

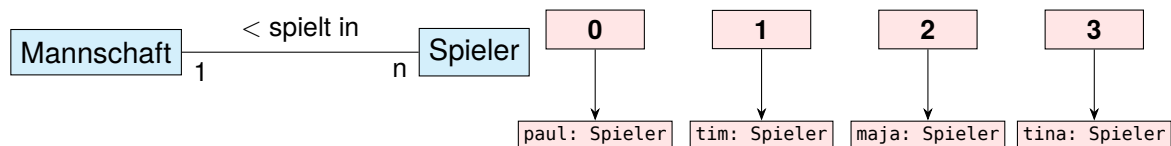
Aufgabe 3

a) Weise allen Zellen zufällige Werte zu (Wiederholung mit fester Anzahl). **b)** 100 Zellen, jede mit der Summe zweier Würfel.

Was änderst du an der Schleife aus a), damit sie bei jeder Feldgröße stimmt?

Bedingung $i < \text{zahlenFeld.length}$ statt einer festen Zahl.

1:n-Beziehung als Sammlung umsetzen – Arrays mit Referenzen



Auch Referenzen können in Arrays verwaltet werden. Dabei verweist jede Zelle auf ein

Objekt der anderen Klasse, hier: Spieler.

Beachte: Alle Elemente eines Feldes müssen denselben Datentyp besitzen, dieser kann aber beliebig gewählt werden: auch boolean, String, Rectangle, ...

Syntaxübersicht

<Sichtbarkeit> <Datentyp>[] <Feldname>; Deklaration: [] nach dem Datentyp macht ein Feld daraus.
 <Feldname> = new <Datentyp>[<Größe>]; Erzeugung mit fester Größe (nicht veränderbar).
 <Feldname>[<Index>] Zugriff auf die Zelle an dieser Position.

Beispiele:

lesen: int eineZahl = zahlenFeld[0];
 Methode aufrufen: kreisFeld[0].tuEtwas();

schreiben: zahlenFeld[0] = eineZahl;
 Attribut: kreisFeld[0].xPosition = 100;

Mehrdimensionale Arrays

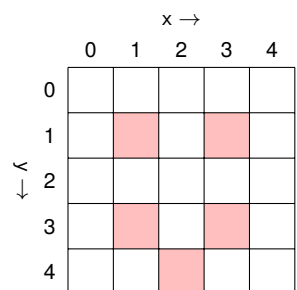
Arrays können mehrere Dimensionen haben und so eine

Tabelle (Raster) aufspannen.

Deklaration und Erzeugung: Rectangle[][] fuellung

fuellung = new Rectangle[5][5];

Zugriff auf Spalte x, Zeile y: fuellung[x][y]



Aufgabe 4 Projekt Mosaik

a) TODO 1 und 2 in Mosaik1: zweidimensionales Array passender Größe erzeugen und in neuesMosaik() füllen.

b) Auf Mosaik2 wechseln, TODO 3–5 (Smiley). Markiere die Smiley-Zellen fuellung[x][y] im Raster oben. (z. B. [1][1], [3][1], [1][3], [2][4], [3][3])